

Milestone 2 — Klasteryzacja danych

- **Autorzy:** Mateusz Świątek, Maciej Mężyk, Patryk Skowron
 - **Zbiór:** PCR Lab Data
 - **Źródło:** [Zenodo #11617408](#)
 - **Notebook:** [notebooks/02_m2_event_clustering.ipynb](#)
-

1. Cel

Klasteryzujemy **dane niskopoziomowe** logu, aby odkryć kroki procesu i wyjaśnić outliery:

- jednostką analizy jest **pojedyncze zdarzenie silnika**, nie endpoint;
- klasteryzujemy **surowy payload data** każdego zdarzenia + **pozycję próbki na płytce** — najniższy dostępny poziom danych;
- celem jest **odkrycie procesu z klastrów** i wyjaśnienie outlierów.

Świadomie **nie** używamy:

- endpointów (adresów serwisów) — to nie są dane niskopoziomowe czynności,
- wyniku PCR (**result/ct/state**) — to ground truth, czyli *wynik* procesu, a nie sposób jego działania,
- agregatów przypadku (**duration_min**) ani **timestamp** jako cechy — czas służy wyłącznie do uporządkowania sekwencji,
- etykiety czynności **activity** — używamy jej **tylko** jako ground truth do walidacji 1:1.

Uwaga o danych

Zbiór to **log procesowy CPEE**, nie telemetria — **nie zawiera fizycznych czujników** (nagłówek deklaruje ontologię W3C SSN/SOSA, ale żadne zdarzenie ich nie używa). Najniższym realnym poziomem danych jest payload pojedynczego wywołania w silniku procesowym i pozycja próbki na płytce 96-dółkowej — i to klasteryzujemy.

Przegląd całego payloadu potwierdza, że poza **sygnaturą argumentów** i **pozycją na płytce** log nie zawiera innych klastrowalnych zmiennych niskopoziomowych: pozostałe pola to identyfikatory (**pid**, **sampleid**, **plateid** — bez semantyki klastrowania), pole puste albo dane wykluczone powyżej (endpointy, ground truth, czas agregatu). Dobór cech jest więc **wyczerpujący**, nie arbitralny.

2. Dane wejściowe — typy i statystyki

Źródło: data/processed/pcr_events_rich.parquet

Metryka	Wartość
Wszystkich zdarzeń silnika	317 905
Przypadków	6 339
Zdarzenia <code>activity/calling</code> (jednostka klasteryzacji)	51 209
Czynności (ground truth)	28

Mikro-stany silnika (`cpee_lifecycle`): `activity/done` (82 890), `dataelements/change` (66 363), `activity/receiving` (58 330), `activity/calling` (51 209), `state/change`, `gateway/join`, `task/instantiation`, ... Payload biznesowy niosą zdarzenia `activity/calling` (moment wywołania czynności z argumentami) — i tylko one są jednostką klasteryzacji. Mikro-stany `receiving/done` nie niosą payloadu, a `dataelements/change/state/change` zawierają wynik PCR (ground truth), więc je pomijamy.

Przykłady danych wejściowych do klasteryzacji (payload + pozycja):

activity (GT)	data_vars (payload)	position	pid
timeout	<code>duration</code>	—	—
Match patient data	<code>pid,sampleid</code>	—	6
Wait for plate validation	<code>id,ttl,delete</code>	—	—
Receive sample state	<code>pid,sampleid,plateid,position</code>	50	13
Callback timeout	<code>stop</code>	—	—

Każda czynność ma charakterystyczną **sygnaturę argumentów** — to jest sygnał niskopoziomowy, na którym pracujemy.

3. Macierz cech

Cechy zdarzenia = **struktura wywołania** (które argumenty są przekazywane, kodowanie one-hot — w tym argumenty czasowe payloadu `duration/ttl`) + **pozycja na płytce** (skalowana wartość + flaga obecności). Z payloadu wykluczamy:

- zmienne **endpointowe/konfiguracyjne** (`timeout-url`, `subprocess`, `receive`, `send`, `correlator`, `notify`, `url`, `endpoints`, `behavior`, `customization`, `init`),
- **ground truth** (`result`, `state`, `ct`),

- **bookkeeping** modelu (info, creator, author, modeltype, ...).

Wynik: **51 209 zdarzeń** $\times$ **17 cech**: attributes, createdids, delete, duration, finishids, id, level, message, pid, plateid, position, sampleid, stop, ttl, value, position_scaled, has_position.

4. Redukcja wymiarowości i klasteryzacja

Liczbę klastrów dobrano metodą sylwetki (KMeans, $k = 3..12$):

k	3	5	7	9	10	11	12
silhouette	0.37	0.52	0.69	0.86	0.86	0.88	0.92

Wybrano $k = 12$ (silhouette **0.917**) — wysoka wartość wynika z tego, że sygnatury argumentów są dyskretne i dobrze rozdzielone.

4.1 PCA zdarzeń

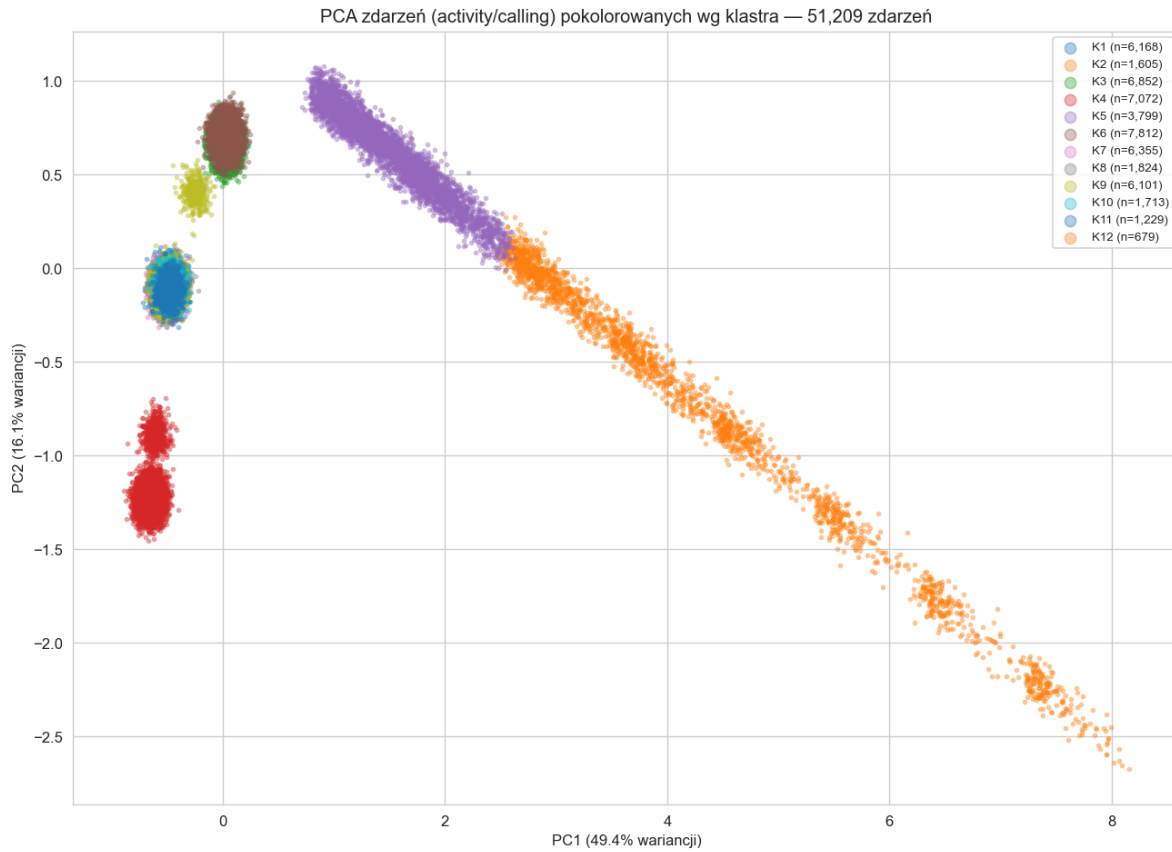


Figure 1: PCA zdarzeń wg klastra

Rzutujemy **51 209 zdarzeń**, kolor = odkryty klaster. Widać wyraźnie rozdzielone grupy; ukośna „smuga” (klastry K2/K5/K12) to czynność *Receive sample state* rozłożona wg **pozycji na płycie** — cecha przestrzenna tworzy ciągły gradient.

5. Charakterystyka i nazwanie klastrów

Klaster	n	Sygnatura argumen- tów	Poz. (mediana)	Dominująca czynność (GT)	Czystość
K1	6 168	<code>duration</code>	—	<code>timeout</code>	100%
K2	1 605	<code>pid,plateid,position,sampleid</code>	35	<code>Receive sample state</code>	100%
K3	6 852	<code>pid,plateid</code>	—	<code>Wait for sample</code>	92%
K4	7 072	<code>delete,id,ttl</code>	—	<code>Wait for plate validation</code>	87%
K5	3 799	<code>pid,plateid,position,sampleid</code>	40	<code>Receive sample state</code>	100%
K6	7 812	<code>pid,sampleid</code>	—	<code>Match patient data</code>	79%
K7	6 355	<code>attributes</code>	—	<code>Spawn per sample flow</code>	97%
K8	1 824	(bez payloadu)	—	<code>Sleep</code>	94%
K9	6 101	<code>stop</code>	—	<code>Callback timeout</code>	95%
K10	1 713	<code>createdids,finishids</code>	—	<code>Check for unfinished Plates</code>	100%
K11	1 229	<code>level,message</code>	—	<code>Send notification</code>	100%
K12	679	<code>pid,plateid,position,sampleid</code>	78	<code>Receive sample state</code>	100%

Obserwacje:

- Każdy klaster ma **jednoznaczny sygnaturę argumentów** odpowiadającą realnej czynności.
- **K1** (`duration` = 25 200 s) to czynność `timeout`.
- **K8** (, bez payloadu) to `Sleep` — jedyna czynność bez żadnych argumentów biznesowych pozostaje nierozróżnialna od „pustki”; to granica informacyjna logu, nie błąd.

- **K2/K5/K12** to ta sama czynność *Receive sample state*, rozdzielona przez **pozycję na płytce** (mediana 9 / 35 / 73) — odkrycie struktury przestrzennej próbek.

6. Porównanie 1:1 z ground truth

Czy klastry odkryte **wyłącznie z payloadu** odtwarzają realne kroki procesu? (Etykieta activity **nie była** cechą.)

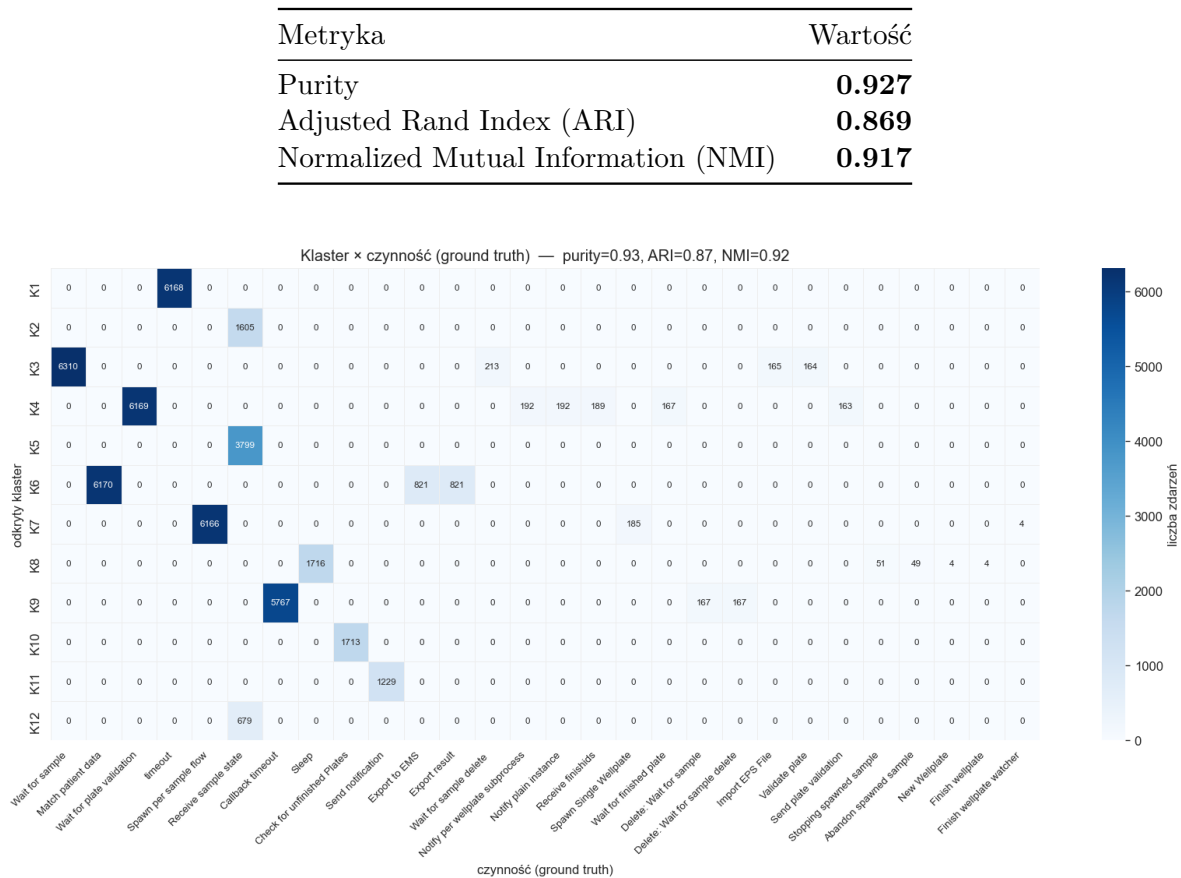


Figure 2: Klastry × czynność

Macierz pomyłek ma strukturę blokową: każdy klastor koncentruje się na jednej czynności. Payload wystarcza, by **bez nadzoru** odtworzyć kroki procesu — co potwierdza sensowność danych wejściowych.

7. Sekwencje klastrów w czasie i graf przejść

Dla każdego przypadku zdarzenia uporządkowano po czasie → sekwencja klastrów (czas służy **tylko** do uporządkowania). Najczęstsze warianty:

Liczność	Sekwencja klastrów
1 124	K1 → K6 → K4 → K5 → K9
907	K1 → K4 → K6 → K5 → K9
465	K1 → K6 → K4 → K2 → K9
351	K1 → K4 → K6 → K2 → K9

Tłumacząc nazwami: **timeout** → **Match patient data** → **Wait for plate validation** → **Receive sample state** → **Callback timeout** — to odtworzony, niskopoziomowo, główny przepływ procesu próbki.

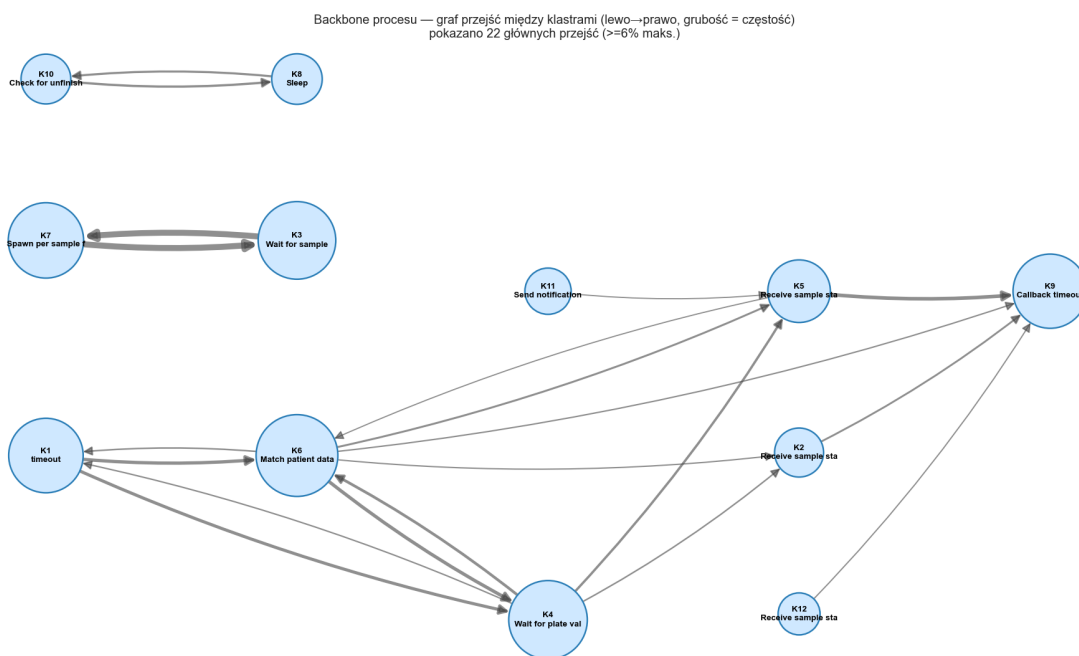


Figure 3: Graf przejść klastrów

8. Outliery z perspektywy odkrytego procesu

Outlier definiujemy **procesowo**: przypadek, którego sekwencja klastrów zawiera **rzadkie przejścia** (spoza głównego grafu), a nie po prostu długi czas.

- Przypadków z rzadkimi przejściami: **57 (0.9%)**.
- Najrzadsze przejścia (np. $K3 \rightarrow K1$, $K1 \rightarrow K3$, $K7 \rightarrow K7$, pętle) wskazują nietypowe kolejności kroków.
- Odniesienie (nie kryterium): mediana `duration_min` dla outlierów procesowych = **408 min** vs **175 min** dla wszystkich — anomalie strukturalne pokrywają się z dłuższym czasem.

W przeciwieństwie do poprzedniej wersji outlier jest **wyjaśniony odkrytym procesem** (nietypowa ścieżka klastrów), a nie zdefiniowany z góry przez `duration`.

9. Wnioski

1. **Dane niskopoziomowe są sensowne.** Surowy payload wywołań CPEE + pozycja na płytce pozwala — bez użycia etykiet — odtworzyć kroki procesu (Purity 0.93, ARI 0.87, NMI 0.92).
2. **Klastry = czynności procesu.** 12 klastrów odpowiada realnym czynnościom; sygnatura argumentów jest „odciskiem palca” czynności.
3. **Odkrycie przestrzenne.** *Receive sample state* dzieli się wg pozycji na płytce (K2/K5/K12) — struktura niewidoczna na poziomie endpointów.
4. **Granica danych.** Jedynie *Sleep* nie niesie żadnego payloadu i tworzy „pusty” klaster K8.
5. **Od klastrów do procesu i outlierów.** Sekwencje klastrów odtwarzają główny przepływ, a rzadkie przejścia wskazują outlierzy procesowe.
6. **Brak telemetrii.** Zbiór nie zawiera fizycznych czujników; przyjęliśmy najniższy dostępny poziom danych (payload zdarzenia), co wprost odpowiada na feedback.